



09-DARS · 14 TADAN

# Express.js va REST API

Express asoslari, marshrutlash, middleware zanjiri, REST API dizayni, validatsiya, xatolarni markazlashgan boshqarish, CORS va loyiha tuzilmasi (MVC).

O'rta · ~140 daqiqa

**Express** — Node.js uchun eng mashhur minimalistik web-freymvork. U oldingi darsda qo'lda yozgan narsalarimizni — marshrutlash, JSON'ni o'qish, statik fayllar, xatolarni boshqarish — oddiy va tushunarli API bilan beradi. Bu darsda frontend ilovalar "gaplashadigan" haqiqiy **REST API** qurishni o'rganamiz.

## Birinchi Express ilova

Terminal

```
mkdir kitob-api && cd kitob-api
npm init -y
npm pkg set type=module
npm install express
```

src/server.js

```
import express from 'express';

const app = express();
app.use(express.json()); // JSON tanani avtomatik o'qish (req.body)

app.get('/', (req, res) => {
  res.send('Kitob API ishlayapti 📖');
});

app.get('/api/salom/:ism', (req, res) => {
  res.json({ xabar: `Salom, ${req.params.ism}!` });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`🚀 http://localhost:${PORT}`));
```

8-darsdagi 35 qatorlik `http` serverini Express 10 qatorda bajardi. Endi har bir qismni chuqurroq ko'ramiz.

## Marshrutlash (Routing)

Marshrut = **HTTP metod** + **yo'l** + **ishlovchi funksiya(lar)**.

JavaScript

```
app.get('/api/books', handler); // o'qish
app.post('/api/books', handler); // yaratish
app.put('/api/books/:id', handler); // to'liq almashtirish
app.patch('/api/books/:id', handler); // qisman yangilash
app.delete('/api/books/:id', handler); // o'chirish
app.all('/api/secret', handler); // istalgan metod

// Bitta yo'l uchun bir nechta metod
app.route('/api/books/:id')
  .get(getBook)
  .patch(updateBook)
  .delete(deleteBook);
```

## So'rov (req) obyektidan ma'lumot olish

JavaScript

```
// GET /api/users/42/orders?status=paid&page=2
app.get('/api/users/:userId/orders', (req, res) => {
  req.params;           // { userId: '42' }           - yo'l parametrlari
  (doim satr!)
  req.query;             // { status: 'paid', page: '2' } - query parametrlar
  (doim satr!)
  req.body;              // POST/PUT tanasi (express.json() kerak)
  req.headers;           // barcha sarlavhalar (kichik harfda)
  req.get('authorization');
  req.method;            // 'GET'
  req.path;              // '/api/users/42/orders'
  req.ip;                // mijoz IP manzili
  req.cookies;           // cookie-parser middleware bilan
});
```

## Javob (res) usullari

JavaScript

```
res.json({ id: 1 });           // JSON + Content-Type avtomatik
res.status(201).json(newBook); // status kod + JSON
res.status(204).end();          // tanasiz javob
res.send('<h1>HTML</h1>');      // matn/HTML
res.sendFile(path.join(dir, 'a.pdf'));
res.download('/files/hisobot.xlsx'); // yuklab olish oynasi
res.redirect(301, '/yangi-manzil');
res.set('Cache-Control', 'no-store');
res.cookie('theme', 'dark', { httpOnly: true, maxAge: 86400000 });
```



Bitta so'rovga **faqat bitta** javob yuboriladi. `res.json()` dan keyin yana `res.send()` chaqirilsa — "Cannot set headers after they are sent" xatosi. Shuning uchun javobdan keyin `return` qilish odatiy: `if (!book) return res.status(404).json(...)`.

## Router — marshrutlarni modullarga ajratish

src/routes/books.routes.js

```
import { Router } from 'express';
const router = Router();

router.get('/', (req, res) => { /* barcha kitoblar */ });
router.get('/:id', (req, res) => { /* bitta kitob */ });
router.post('/', (req, res) => { /* yaratish */ });

export default router;
```

src/app.js

```
import booksRouter from './routes/books.routes.js';
import authorsRouter from './routes/authors.routes.js';

app.use('/api/books', booksRouter); // router ichidagi '/' → '/api/books'
app.use('/api/authors', authorsRouter);
```

## Middleware — Express'ning yuragi

**Middleware** — so'rov va javob o'rtasida ishlaydigan funksiya. Express ilovasi — bu middleware'lar **zanjiri**: so'rov ularning har biridan ketma-ket o'tadi.



Har bir middleware so'rovni o'zgartirishi, javob qaytarib zanjirni to'xtatishi yoki next() bilan keyingisiga uzatishi mumkin

## JavaScript

```
function middleware(req, res, next) {  
  // 1. req/res ni o'qish yoki o'zgartirish  
  // 2. va keyin:  
  next();           // → keyingi middleware'ga o'tish  
  // yoki: res.status(401).json(...) – zanjirni to'xtatish  
  // yoki: next(err)           – xato ishlovchisiga o'tish  
}
```

## Middleware turlari

### JavaScript

```
// 1. Ilova darajasida – barcha so'rovlar uchun  
app.use(express.json({ limit: '1mb' }));           // JSON tana  
app.use(express.urlencoded({ extended: true }));   // HTML forma tanasi  
app.use(express.static('public'));                 // statik fayllar  
(public/index.html → /)  
  
// 2. Yo'l bo'yicha  
app.use('/api', apiRateLimiter);  
  
// 3. Marshrut darajasida – faqat shu marshrut uchun (bir nechta bo'lishi  
mumkin)  
app.delete('/api/books/:id', requireAuth, requireRole('admin'), deleteBook);  
  
// 4. Xato ishlovchi – 4 ta parametr (err, req, res, next). Doim ENG OXIRIDA  
app.use((err, req, res, next) ⇒ {  
  res.status(500).json({ message: 'Server xatosi' });  
});
```



### Tartib muhim!

Middleware'lar **yozilgan tartibda** bajariladi. `express.json()` marshrutlardan keyin yozilsa, `req.body` `undefined` bo'ladi. 404 ishlovchisi barcha marshrutlardan keyin, xato ishlovchisi — eng oxirida.

# Mashhur tayyor middleware'lar

| Paket                           | Vazifasi                                                 |
|---------------------------------|----------------------------------------------------------|
| <code>cors</code>               | Boshqa domendagi frontendga API'dan foydalanishga ruxsat |
| <code>helmet</code>             | Xavfsizlik sarlavhalarini o'rnatadi                      |
| <code>morgan</code>             | So'rovlarni log qilish                                   |
| <code>express-rate-limit</code> | Bir IP'dan so'rovlar sonini cheklash                     |
| <code>cookie-parser</code>      | Cookie'larni <code>req.cookies</code> ga o'qish          |
| <code>multer</code>             | Fayl yuklash ( <code>multipart/form-data</code> )        |
| <code>compression</code>        | Javoblarni gzip bilan siqish                             |

# REST API dizayni

**REST (Representational State Transfer)** — API'ni loyihalash uslubi. Asosiy g'oya: hamma narsa — **resurs** (kitob, foydalanuvchi, buyurtma), har bir resurs **URL** bilan aniqlanadi, ular ustidagi amallar **HTTP metodlari** bilan ifodalanadi.

| Amal             | Metod va URL                          | Muvaffaqiyatli javob                         |
|------------------|---------------------------------------|----------------------------------------------|
| Ro'yxat          | <code>GET /api/books</code>           | <code>200</code> + massiv                    |
| Bittasini olish  | <code>GET /api/books/42</code>        | <code>200</code> + obyekt / <code>404</code> |
| Yaratish         | <code>POST /api/books</code>          | <code>201</code> + yaratilgan obyekt         |
| To'liq yangilash | <code>PUT /api/books/42</code>        | <code>200</code>                             |
| Qisman yangilash | <code>PATCH /api/books/42</code>      | <code>200</code>                             |
| O'chirish        | <code>DELETE /api/books/42</code>     | <code>204</code>                             |
| Ichki resurs     | <code>GET /api/authors/7/books</code> | <code>200</code>                             |

## Yaxshi API qoidalari

- URL'da **otlar**, ko'plikda: `/api/books` ✓, `/api/getBooks` ✗, `/api/book/delete/5` ✗.
- Kichik harflar va defis: `/api/order-items`.
- Filtrlash, saralash, sahifalash — query orqali: `/api/books?genre=roman&sort=-price&page=2&limit=20`.
- To'g'ri status kodlar (1-dars jadvali).
- Xatolar **bir xil formatda**: `{ "error": { "message": "...", "details": [...] } }`.
- Versiyalash: `/api/v1/books` — API'ni buzuvchi o'zgarishlar kiritilganda eski mijozlar ishlashda davom etadi.
- Sahifalangan javobda meta-ma'lumot: `{ data: [...], meta: { page, limit, total } }`.

## Validatsiya

Mijozdan kelgan har qanday ma'lumot — **ishonchsiz**. Validatsiyasiz API bazaga noto'g'ri ma'lumot yozadi, xatolar bilan "yiqiladi" va hujumlarga ochiq bo'ladi. Qo'lda tekshirish mumkin, lekin **Zod** kabi sxema kutubxonalari qulayroq:

### Terminal

```
npm install zod
```

```
import { z } from 'zod';

const bookSchema = z.object({
  title: z.string().trim().min(1, 'Nom majburiy').max(200),
  author: z.string().trim().min(2),
  price: z.number().int().positive('Narx musbat bo\'lishi kerak'),
  year: z.number().int().min(1450).max(new Date().getFullYear()).optional(),
  genres: z.array(z.enum(['roman', 'ilmiy', 'bolalar',
    'dasturlash']))).default([]),
});

const result = bookSchema.safeParse(req.body);
if (!result.success) {
  // result.error.issues → [{ path: ['price'], message: 'Narx musbat bo\'lishi
  kerak' }, ...]
}
const cleanData = result.data; // tozalangan, standart qiymatlar qo'shilgan,
ortiqcha maydonlar olib tashlangan

// PATCH uchun — barcha maydonlar ixtiyoriy
const bookPatchSchema = bookSchema.partial();
```

## Xatolarni markazlashgan boshqarish

Har bir marshrutda `try/catch` yozib, xatoni qo'lda formatlash — takrorlanuvchi kod. To'g'ri yondashuv: marshrutlar xatoni **tashlaydi (throw)**, bitta xato middleware'i ularni **bir xil formatda** javobga aylantiradi.

```
export class HttpError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
  static badRequest(msg = 'Noto\'g\'ri so\'rov', details) { return new
HttpError(400, msg, details); }
  static unauthorized(msg = 'Avtorizatsiya talab qilinadi') { return new
HttpError(401, msg); }
  static forbidden(msg = 'Ruxsat yo\'q') { return new HttpError(403, msg); }
  static notFound(msg = 'Topilmadi') { return new HttpError(404, msg); }
  static conflict(msg) { return new HttpError(409, msg); }
}
```

```
import { HttpError } from '../utils/errors.js';

export function notFound(req, res, next) {
  next(HttpError.notFound(`Marshrut topilmadi: ${req.method}
${req.originalUrl}`));
}

// eslint-disable-next-line no-unused-vars
export function errorHandler(err, req, res, next) {
  // Noto'g'ri JSON (express.json() tashlaydi)
  if (err.type === 'entity.parse.failed') {
    err = HttpError.badRequest('So\'rov tanasidagi JSON noto\'g\'ri');
  }

  const status = err.status ?? 500;
  if (status ≥ 500) console.error(err); // kutilmagan xatolarni log qilamiz

  res.status(status).json({
    error: {
      message: status ≥ 500 && process.env.NODE_ENV === 'production'
        ? 'Ichki server xatosi' // 🚫 production'da ichki
        : err.message,          tafsilotlarni oshkor qilmaymiz
      ...(err.details && { details: err.details }),
    },
  });
}
```



### Express 5 va async xatolar

Express 5 da `async` marshrut ichida tashlangan xato yoki rad etilgan Promise avtomatik ravishda xato middleware'iga uzatiladi. Express 4 da esa buning uchun har bir handler'ni o'rash yoki `express-async-errors` paketini ishlatish kerak edi.

# CORS

Frontend `http://localhost:5173` da, API `http://localhost:3000` da ishlasa, brauzer so'rovni bloklaydi (6-dars). Ruxsatni server beradi:

## JavaScript

```
import cors from 'cors';

app.use(cors({
  origin: ['http://localhost:5173', 'https://kitoblar.uz'], // faqat ishonchli
  manbalar
  credentials: true, // cookie
  yuborishga ruxsat
}));
// ⚠ origin: '*' – ochiq API'lardan tashqari tavsiya etilmaydi
```

## Loyiha tuzilmasi

Hamma kodni bitta faylga yozish 100 qatorgacha ishlaydi. Katta loyihalarda **mas'uliyatlarni ajratish** kerak:

```

kitob-api/
├── src/
│   ├── server.js           # faqat ishga tushirish (listen)
│   ├── app.js             # Express ilova: middleware'lar va router'lar
│   ├── config.js          # muhit o'zgaruvchilari bir joyda
│   ├── routes/            # URL → controller bog'lanishi
│   │   └── books.routes.js
│   ├── controllers/       # HTTP qatlami: req'dan o'qish, res'ga yozish
│   │   └── books.controller.js
│   └── services/          # biznes-mantiq (HTTP haqida hech narsa
bilmaydi)
│   └── books.service.js
│   ├── db/               # ma'lumotlar bilan ishlash
│   ├── middleware/       # auth, validate, errorHandler
│   ├── schemas/          # Zod sxemalari
│   └── utils/
├── tests/
├── .env
├── .env.example
├── .gitignore
└── package.json

```

**Nega bunday?** Controller "qanday so'raldi" bilan, service "nima qilish kerak" bilan, db qatlami "qayerda saqlanadi" bilan shug'ullanadi. JSON fayldan PostgreSQL'ga o'tganda faqat db qatlami o'zgaradi; service'ni HTTP'siz test qilish mumkin. `app.js` va `server.js` ning ajratilishi esa testlarda ilovani portni band qilmasdan ishga tushirishga imkon beradi (14-dars).

## Amaliy misollar

### MISOL 1 Kitoblar do'koni uchun to'liq CRUD REST API

8-darsdagi `JsonStore` ustiga qurilgan, yuqoridagi tuzilmaga amal qilgan to'liq API. Filtrlash, qidiruv, saralash va sahifalash ham bor.

## Terminal

```
npm install express zod cors helmet morgan
```

src/schemas/book.schema.js

```
import { z } from 'zod';

export const bookSchema = z.object({
  title: z.string().trim().min(1).max(200),
  author: z.string().trim().min(2).max(100),
  price: z.number().int().positive(),
  stock: z.number().int().min(0).default(0),
  genre: z.enum(['roman', 'ilmiy', 'bolalar', 'dasturlash', 'tarix']),
  year: z.number().int().min(1450).max(new
Date().getFullYear()).optional(),
});

export const bookPatchSchema = bookSchema.partial();

export const listQuerySchema = z.object({
  q: z.string().trim().optional(),
  genre: bookSchema.shape.genre.optional(),
  minPrice: z.coerce.number().int().min(0).optional(), // coerce:
'5000' → 5000
  maxPrice: z.coerce.number().int().min(0).optional(),
  sort: z.enum(['price', '-price', 'title', '-createdAt']).default('-
createdAt'),
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(10),
});
```

src/middleware/validate.js

```
import { HttpError } from '../utils/errors.js';

// Sxema bo'yicha req.body / req.query ni tekshiruvchi middleware
"fabrikasi"
export const validate = (schema, source = 'body') => (req, res, next) => {
  const result = schema.safeParse(req[source]);
  if (!result.success) {
    const details = result.error.issues.map(i => ({ field:
i.path.join('.'), message: i.message }));
    return next(HttpError.badRequest('Validatsiya xatosi', details));
  }
  req.validated = { ...req.validated, [source]: result.data };
  next();
};
```

```

import { JsonStore } from '../db/jsonStore.js';
import { HttpError } from '../utils/errors.js';

const store = new JsonStore('data/books.json');

const SORTERS = {
  price: (a, b) => a.price - b.price,
  '-price': (a, b) => b.price - a.price,
  title: (a, b) => a.title.localeCompare(b.title),
  '-createdAt': (a, b) => b.createdAt.localeCompare(a.createdAt),
};

export async function listBooks({ q, genre, minPrice, maxPrice, sort,
page, limit }) {
  const needle = q?.toLowerCase();
  const all = await store.all({
    filter: (b) =>
      (!needle || b.title.toLowerCase().includes(needle) ||
b.author.toLowerCase().includes(needle)) &&
      (!genre || b.genre === genre) &&
      (minPrice === undefined || b.price ≥ minPrice) &&
      (maxPrice === undefined || b.price ≤ maxPrice),
    sort: SORTERS[sort],
  });
  const start = (page - 1) * limit;
  return {
    data: all.slice(start, start + limit),
    meta: { page, limit, total: all.length, pages: Math.ceil(all.length /
limit) },
  };
}

export async function getBook(id) {
  const book = await store.get(id);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function createBook(data) {
  const duplicate = await store.all({
    filter: b => b.title.toLowerCase() === data.title.toLowerCase() &&
b.author === data.author,
  });

```

```

});
if (duplicate.length) throw HttpError.conflict('Bu kitob allaqachon mavjud');
return store.create(data);
}

export async function updateBook(id, patch) {
  const book = await store.update(id, patch);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function deleteBook(id) {
  if (!(await store.remove(id))) throw HttpError.notFound('Kitob topilmadi');
}

```

src/controllers/books.controller.js

```

import * as service from '../services/books.service.js';

export const list = async (req, res) => res.json(await service.listBooks(req.validated.query));
export const getOne = async (req, res) => res.json(await service.getBook(req.params.id));
export const create = async (req, res) => res.status(201).json(await service.createBook(req.validated.body));
export const update = async (req, res) => res.json(await service.updateBook(req.params.id, req.validated.body));
export const remove = async (req, res) => {
  await service.deleteBook(req.params.id);
  res.status(204).end();
};

```

src/routes/books.routes.js

```
import { Router } from 'express';
import * as ctrl from '../controllers/books.controller.js';
import { validate } from '../middleware/validate.js';
import { bookSchema, bookPatchSchema, listQuerySchema } from
'../schemas/book.schema.js';

const router = Router();

router.get('/', validate(listQuerySchema, 'query'), ctrl.list);
router.get('/:id', ctrl.getOne);
router.post('/', validate(bookSchema), ctrl.create);
router.patch('/:id', validate(bookPatchSchema), ctrl.update);
router.delete('/:id', ctrl.remove);

export default router;
```

src/app.js

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import booksRouter from './routes/books.routes.js';
import { notFound, errorHandler } from './middleware/errorHandler.js';

export const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL ?? 'http://localhost:5173'
})));
app.use(morgan('dev'));
app.use(express.json({ limit: '100kb' })));

app.get('/api/health', (req, res) => res.json({ status: 'ok', uptime:
process.uptime() }));
app.use('/api/v1/books', booksRouter);

app.use(notFound);
app.use(errorHandler);
```

src/server.js

```
import { app } from './app.js';

const PORT = Number(process.env.PORT) || 3000;
app.listen(PORT, () => console.log(`📖 Kitob API:
http://localhost:${PORT}/api/v1/books`));
```

Sinab ko'ramiz:

Terminal

```
# Yaratish
curl -X POST localhost:3000/api/v1/books -H "Content-Type:
application/json" \
  -d '{"title":"O'tkan kunlar","author":"Abdulla
Qodiriy","price":65000,"stock":10,"genre":"roman","year":1926}'
# → 201 { "id": "c1f...", "title": "O'tkan kunlar", ... }

# Validatsiya xatosi
curl -X POST localhost:3000/api/v1/books -H "Content-Type:
application/json" -d '{"title":"","price":-5}'
# → 400 { "error": { "message": "Validatsiya xatosi", "details": [
#       { "field": "title", "message": "..."},
#       { "field": "author", "message": "..."}, ... ] } }
#   (aniq xabar matni Zod versiyasiga bog'liq; o'zbekcha xabarlarni
sxemada berish mumkin)

# Ro'yxat: qidiruv, filtr, saralash, sahifalash
curl "localhost:3000/api/v1/books?q=qodiriy&genre=roman&sort=-
price&page=1&limit=5"

# Qisman yangilash va o'chirish
curl -X PATCH localhost:3000/api/v1/books/ID -H "Content-Type:
application/json" -d '{"price":59000}'
curl -i -X DELETE localhost:3000/api/v1/books/ID      # → 204
curl -i localhost:3000/api/v1/books/ID                 # → 404
```



curl o'rniga VS Code'ning **REST Client** kengaytmasi, **Thunder Client** yoki **Postman/Insomnia** dasturlaridan foydalanish qulayroq.

src/middleware/common.js

```
import { randomUUID } from 'node:crypto';

// 1. Har bir so'rovga noyob ID – loglarda bitta so'rovning izini kuzatish uchun
export function requestId(req, res, next) {
  req.id = req.get('X-Request-Id') ?? randomUUID();
  res.set('X-Request-Id', req.id);
  next();
}

// 2. Tuzilgan (structured) logger: javob yuborilgandan keyin yozadi
export function logger(req, res, next) {
  const start = process.hrtime.bigint();
  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    const line = {
      time: new Date().toISOString(),
      id: req.id,
      method: req.method,
      url: req.originalUrl,
      status: res.statusCode,
      ms: Math.round(ms * 10) / 10,
      ip: req.ip,
    };
    (res.statusCode ≥ 500 ? console.error : console.log)
    (JSON.stringify(line));
  });
  next();
}

// 3. Sekin so'rovlarni aniqlash
export const slowWarning = (thresholdMs = 1000) => (req, res, next) => {
  const t = setTimeout(() => console.warn(`🐢 Sekin so'rov
  (${thresholdMs}ms): ${req.method} ${req.originalUrl}`), thresholdMs);
  res.on('finish', () => clearTimeout(t));
  next();
};
```

```

// 4. API kaliti bilan himoya (masalan, ichki xizmatlar uchun)
export const requireApiKey = (req, res, next) => {
  const key = req.get('X-API-Key');
  if (!key || key !== process.env.INTERNAL_API_KEY) {
    return res.status(401).json({ error: { message: 'API kaliti noto\'g\'ri' } });
  }
  next();
};

// 5. Juda oddiy xotiradagi rate limiter (o'rganish uchun; productionda express-rate-limit + Redis)
export function rateLimit({ windowMs = 60_000, max = 60 } = {}) {
  const hits = new Map(); // ip -> { count, reset }
  return (req, res, next) => {
    const now = Date.now();
    const entry = hits.get(req.ip) ?? { count: 0, reset: now + windowMs };
    if (now > entry.reset) Object.assign(entry, { count: 0, reset: now + windowMs });
    entry.count++;
    hits.set(req.ip, entry);

    res.set('RateLimit-Remaining', Math.max(0, max - entry.count));
    if (entry.count > max) {
      res.set('Retry-After', Math.ceil((entry.reset - now) / 1000));
      return res.status(429).json({ error: { message: 'Juda ko\'p so\'rov. Birozdan keyin urinib ko\'ring.' } });
    }
    next();
  };
}

```

src/app.js (qo'shimcha)

```
import { requestId, logger, slowWarning, requireApiKey, rateLimit } from
'./middleware/common.js';

app.use(requestId);
app.use(logger);
app.use(slowWarning(800));
app.use('/api', rateLimit({ windowMs: 60_000, max: 100 }));
app.use('/internal', requireApiKey, internalRouter);
```

Log natijasi: {"time":"2026-10-

03T10:00:00.000Z","id":"5b1e...","method":"GET","url":"/api/v1/books","status":200,"ms":4.2,""

JSON formatidagi loglarni keyinchalik monitoring tizimlarida (Grafana, Datadog) qidirish va tahlil qilish oson.

### MISOL 3 Fayl yuklash: kitob muqovasi (multer)

Terminal

```
npm install multer
```

```

import { Router } from 'express';
import multer from 'multer';
import path from 'node:path';
import { randomUUID } from 'node:crypto';
import { HttpError } from '../utils/errors.js';

const ALLOWED = new Map([
  ['image/jpeg', '.jpg'],
  ['image/png', '.png'],
  ['image/webp', '.webp'],
]);

const upload = multer({
  storage: multer.diskStorage({
    destination: 'uploads/covers',
    // 🛒 Foydalanuvchi bergan nomni ISHLATMAYMIZ – tasodifiy nom +
    ishonchli kengaytma
    filename: (req, file, cb) => cb(null, randomUUID() +
ALLOWED.get(file.mimetype)),
  }),
  limits: { fileSize: 2 * 1024 * 1024, files: 1 }, // 2 MB
  fileFilter: (req, file, cb) =>
    ALLOWED.has(file.mimetype)
      ? cb(null, true)
      : cb(HttpError.badRequest('Faqat JPG, PNG yoki WEBP rasm')),
});

const router = Router();

router.post('/books/:id/cover', upload.single('cover'), async (req, res)
=> {
  if (!req.file) throw HttpError.badRequest('Rasm yuborilmadi (maydon
nomi: cover)');
  const url = `/uploads/covers/${req.file.filename}`;
  // await booksService.updateBook(req.params.id, { coverUrl: url });
  res.status(201).json({ url, size: req.file.size });
});

export default router;

```

src/app.js (qo'shimcha)

```
import fs from 'node:fs';
fs.mkdirSync('uploads/covers', { recursive: true });
app.use('/uploads', express.static('uploads', { maxAge: '7d' }));
app.use('/api/v1', uploadRouter);

// errorHandler ichida multer xatolarini ham tushunarli qilamiz:
// if (err.code === 'LIMIT_FILE_SIZE') err = HttpError.badRequest('Fayl 2 MB dan katta');
```

Terminal

```
curl -F "cover=@muqova.jpg" localhost:3000/api/v1/books/ID/cover
# → 201 { "url": "/uploads/covers/0b6e...c1.jpg", "size": 184322 }
```

Frontend tomonida: `const fd = new FormData(); fd.append('cover', input.files[0]);`  
`fetch(url, { method: 'POST', body: fd })`. Production'da fayllar odatda serverning diskida emas, S3 kabi obyekt omborlarida saqlanadi.

#### MISOL 4 API'ni frontend bilan bog'lash: statik sahifa + fetch

Express bitta serverda ham API'ni, ham frontend fayllarni bera oladi. `public/` papkasiga oddiy sahifa qo'shamiz:

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kitob do'koni</title>
  <style>
    body { font-family: system-ui; max-width: 760px; margin: 40px auto;
padding: 0 16px; }
    form { display: grid; grid-template-columns: 2fr 2fr 1fr 1fr auto;
gap: 8px; margin-bottom: 24px; }
    input, select, button { padding: 8px; font: inherit; }
    .book { display: flex; justify-content: space-between; padding: 10px
0; border-bottom: 1px solid #eee; }
    .error { color: #b91c1c; }
  </style>
</head>
<body>
  <h1>📖 Kitoblar</h1>
  <form id="add">
    <input name="title" placeholder="Nomi" required>
    <input name="author" placeholder="Muallif" required>
    <input name="price" type="number" placeholder="Narx" required>
    <select name="genre">
      <option value="roman">Roman</option><option
value="dasturlash">Dasturlash</option>
      <option value="ilmiy">Ilmiy</option><option
value="bolalar">Bolalar</option>
    </select>
    <button>Qo'shish</button>
  </form>
  <p id="msg" class="error"></p>
  <input id="search" placeholder="🔍 Qidirish..."
style="width:100%;margin-bottom:12px">
  <div id="list"></div>

  <script type="module">
    const API = '/api/v1/books'; // bir xil origin — CORS kerak
emas
    const list = document.querySelector('#list');
    const msg = document.querySelector('#msg');

```

```

    async function request(url, options) {
      const res = await fetch(url, {
        ...options,
        headers: { 'Content-Type': 'application/json' },
      });
      if (res.status === 204) return null;
      const data = await res.json();
      if (!res.ok) {
        const details = data.error.details?.map(d => `${d.field}:
        ${d.message}`).join('; ');
        throw new Error(details || data.error.message);
      }
      return data;
    }

    async function load(q = '') {
      const { data, meta } = await request(`${API}?
      limit=50&q=${encodeURIComponent(q)}`);
      list.replaceChildren(...data.map(b => {
        const row = document.createElement('div');
        row.className = 'book';
        const info = document.createElement('span');
        info.textContent = `${b.title} - ${b.author}
        (${b.price.toLocaleString()} so'm)`;
        const del = document.createElement('button');
        del.textContent = '🗑';
        del.onclick = async () => { await request(`${API}/${b.id}`, {
        method: 'DELETE' }); load(q); };
        row.append(info, del);
        return row;
      }));
      if (!meta.total) list.textContent = 'Kitob topilmadi';
    }

    document.querySelector('#add').addEventListener('submit', async (e) =>
    {
      e.preventDefault();
      const body = Object.fromEntries(new FormData(e.target));
      body.price = Number(body.price);
      try {
        await request(API, { method: 'POST', body: JSON.stringify(body)
      });
      e.target.reset();
      msg.textContent = '';
    }
  
```

```

    load();
  } catch (err) {
    msg.textContent = err.message;      // server validatsiya xabarini
    ko'rsatamiz
  }
});

let t;
document.querySelector('#search').addEventListener('input', (e) => {
  clearTimeout(t);
  t = setTimeout(() => load(e.target.value), 300);
});

load();
</script>
</body>
</html>

```

src/app.js (qo'shimcha)

```

app.use(express.static('public'));    // http://localhost:3000 →
public/index.html

```

Tabriklaymiz — bu sizning birinchi **full-stack** ilovangiz: brauzer → fetch → Express → validatsiya → service → fayl ombori va orqaga. Keyingi darslarda ombor o'rnini PostgreSQL egallaydi, autentifikatsiya qo'shiladi, frontend esa React'ga ko'chadi.

## Mashqlar



### Mustaqil ish

1. Kitob API'siga `authors` resursini qo'shing: `GET /api/v1/authors/:id/books` muallifning barcha kitoblarini qaytarsin.
2. `POST /api/v1/orders` yarating: `{ items: [{ bookId, qty }] }` qabul qilsin, `stock` yetariliginı tekshirib, kamaytirsın; yetmasa `409`.
3. Barcha javoblarga `X-Response-Time` sarlavhasini qo'shuvchi middleware yozing.

4. `GET /api/v1/books/stats` — janrlar bo'yicha kitoblar soni, o'rtacha narx va eng qimmat kitob. (Diqqat: bu marshrut `/:id` dan **oldin** e'lon qilinishi kerak — nega?)
5. API hujjatini `README.md` da yozing: har bir endpoint, parametrlar, so'rov va javob namunalari, xato kodlari.

#### ▼ 4-savol javobi

Express marshrutlarni yozilgan tartibda tekshiradi. Agar `/:id` oldin tursa, `/stats` so'rovi unga mos keladi va `id = 'stats'` deb qabul qilinib, "Kitob topilmadi" qaytadi.

## ✓ Xulosa

- Express ilovasi — middleware'lar zanjiri; `next()` boshqaruvni uzatadi, tartib muhim.
- REST: resurslar (otlar, ko'plikda) + HTTP metodlari + to'g'ri status kodlar.
- Mijoz ma'lumotini doim validatsiya qiling (Zod); xatolarni bitta middleware'da bir xil formatda qaytaring.
- Routes → Controllers → Services → DB qatlamlari kodni test qilinadigan va o'zgartirishga oson qiladi.
- `cors`, `helmet`, rate limiting, fayl yuklashda ehtiyotkorlik — xavfsiz API asoslari.

☐ Darsni o'qib bo'ldim